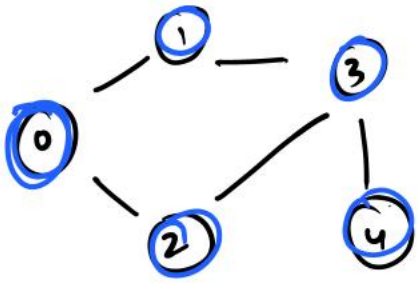
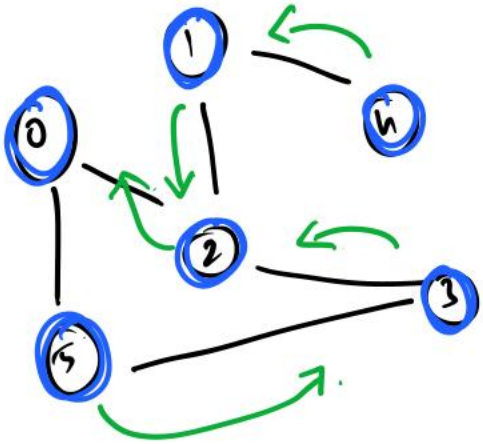
Day 3

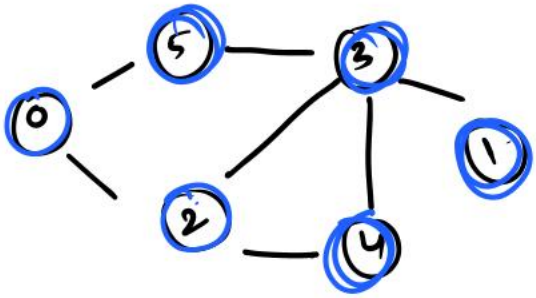
Wednesday, 4 November 2026 10:02 AM



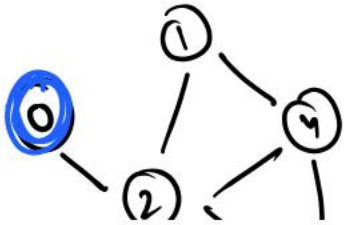
0 1 3 2 4

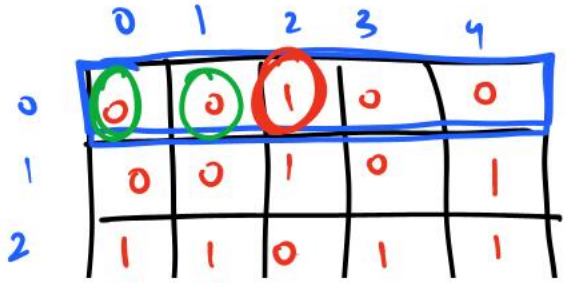


0 2 1 4 3 5

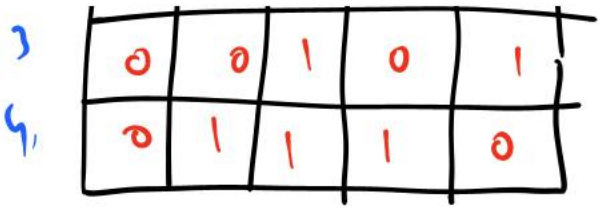


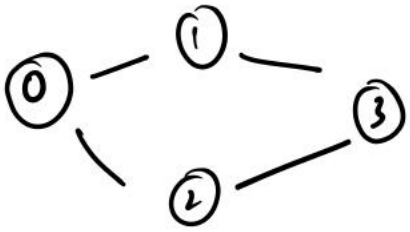
0 2 3 1 4 5







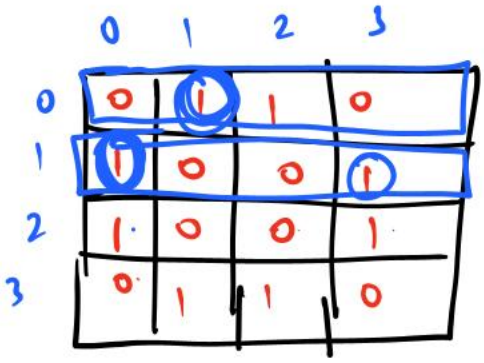


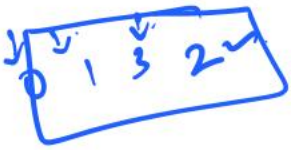


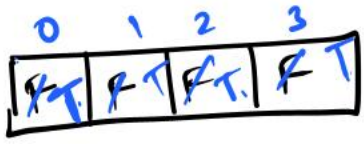
4 ← n

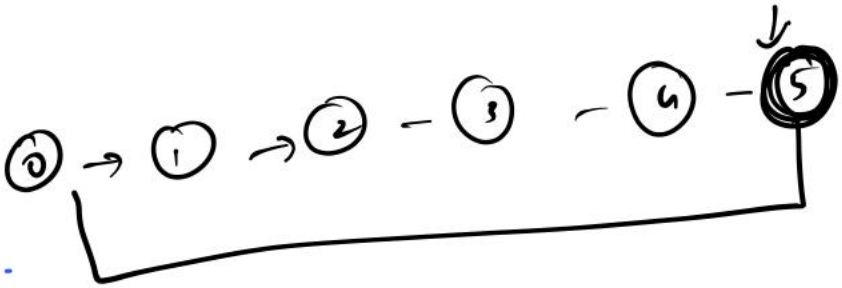
edge → 4

0 1
0 2
1 3
2 3



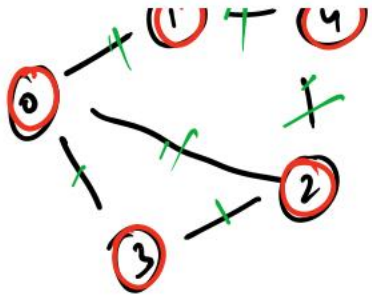


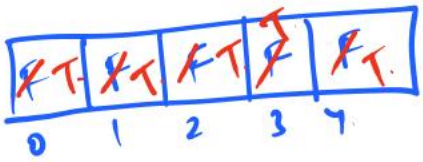


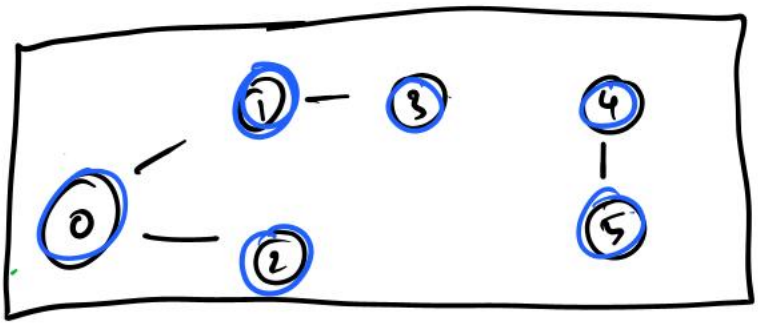






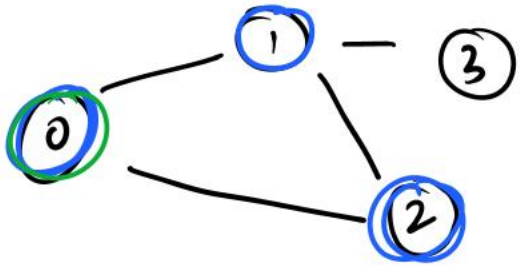


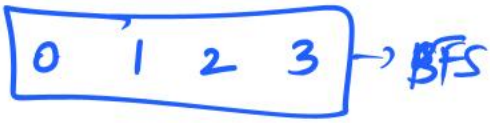




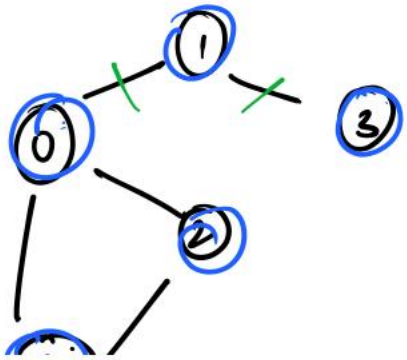
0 1 3 2

4 5





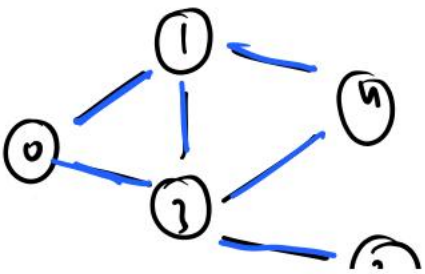




0 1 3 2 4 → DFS

```python
def bfTraversal(adjMatrix) :
    visited = [False] * len(adjMatrix)
    pendingVertices = deque()
    pendingVertices.append(0)
    visited[0] = True
    while pendingVertices :
        vertex = pendingVertices.popleft()
        print(vertex , end = ' ')
        for i in range(len(adjMatrix)) :
            if adjMatrix[vertex][i] == 1 and not visited[i] :
                pendingVertices.append(i)
                visited[i] = True
```

Exploring unvisited → neighbors of vertex.

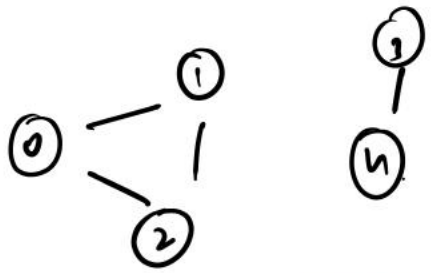


v1 = 0 v2 = 2.

Return any one { 0→3→2 ; 0→1→3→2 ; 0→1→4→3→2 }

Find path btw 2 vertices

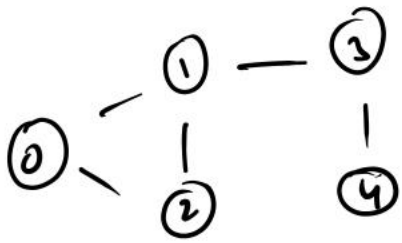
o/p → list of vertices



v1 = 0 v2 = 3

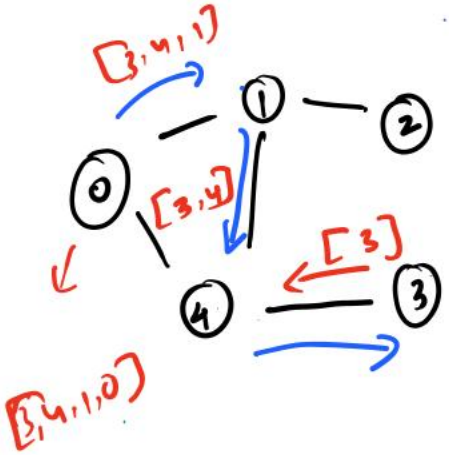
X

None — No path exists



v1 = 0 v2 = 3 ✓

[0, 1, 3] DFS



[3, 4, 1]

[3, 4]

[3]

[3, 4, 1, 0]

0 3

↑ ↑

start dest

→ Move n−1 disks → source, destination, helper

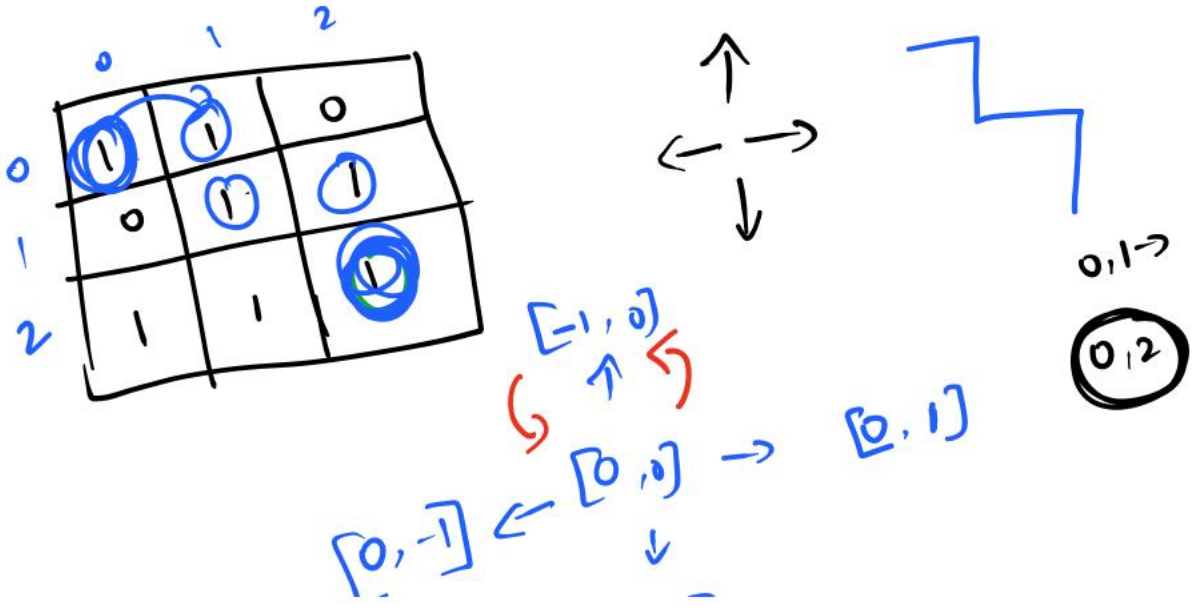
③ → Print (source , " " , destination

↘ Move n-1 disks → helper , source , destination

```
def tower(disks, source, helper, destination):
    if disks == 1 :
        print(source , ' ', destination)
        return
    tower(disks - 1, source, destination, helper)
    print(source , ' ', destination)
    tower(disks - 1, helper, source, destination)

disks = int(input('Enter no of disks : '))
tower(disks, 's', 'h', 'd')
```

T.C → $2^n - 1$ ≈ 2^n



[1, 0]

```python
def helper(maze, i, j, visited) :
    if i < 0 or i == len(maze) or j < 0 or j == len(maze[0]) or maze[i][j] == 0 or
    visited[i][j] :
        return False
    if i == len(maze) - 1 and j == len(maze[0]) - 1 :
        return True
    visited[i][j] = True
    if helper(maze, i - 1, j, visited) or helper(maze, i, j + 1, visited) or helper
    (maze, i + 1, j, visited) or helper(maze, i, j - 1, visited) :
        return True
    return False

def solveMaze(maze, m, n) :
    visited = [[False] * n for _ in range(m)]
    return helper(maze, 0, 0, visited)

maze = [
    [1,1,0],
    [0,0,1],
    [0,0,1]
]
print('Is path exist -> ', solveMaze(maze, 3, 3))
```

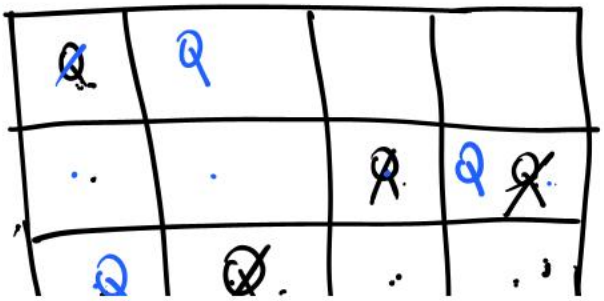
Invalid cell

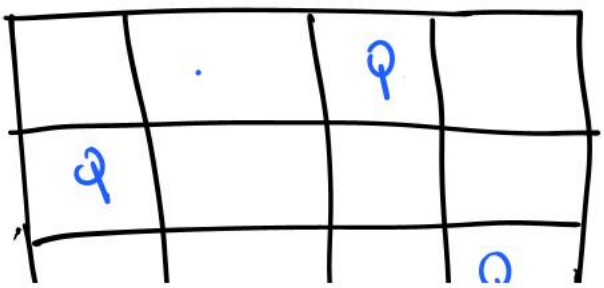
Top

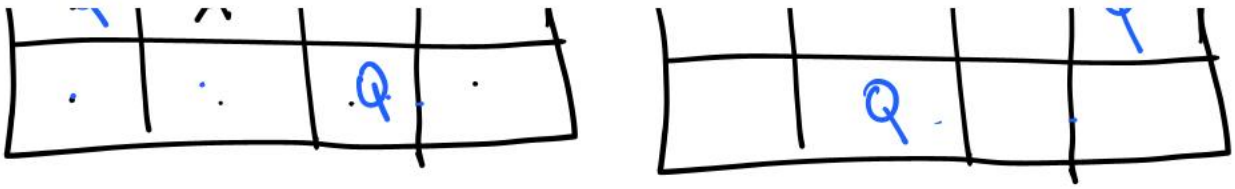
Right

left

down





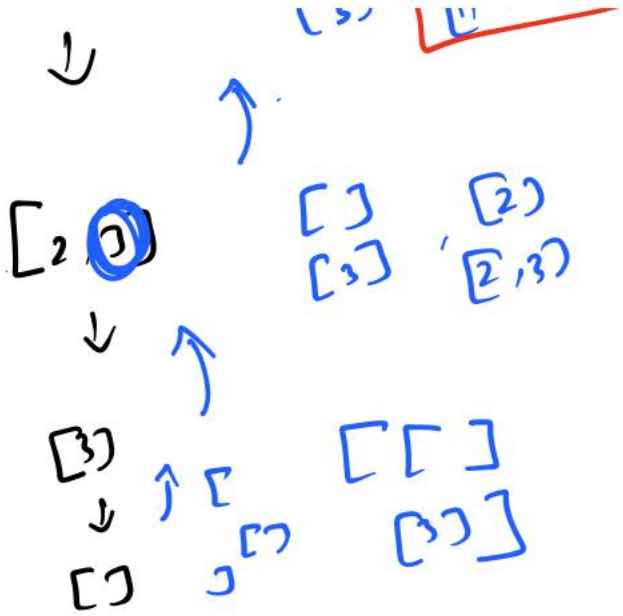


[1,2] → [] [1] [2] [1,2] → 4

[1,2,3] → [] [1] [2] [3] [1,2] [1,3] [2,3] [1,2,3] → 8

2^n

[1,2,3] [] [2] [2,3] [1] [1,3] [2,3]



```python
class Solution:
    def subsets(self, nums: list[int]) -> list[list[int]]:
        def helper(i) :
            if i == len(nums) :
                return [[]]
            smallOutput = helper(i + 1)
            output = []
            for row in smallOutput :
                output.append(row)
                output.append([nums[i]] + row)
            return output
        return helper(0)
```

T.C -> $O(n+2^n)$

S.C -> $O(n+2^n)$

call stack

2D array size